22BCON1068 Artificial Intelligence 27/11/24

Assignment - 05

PAGE NO.: 30
DATE: / /

• Sec-A (2 x 5 = 10)

1. How do you prove a problem in NP-hard.

Ans To prove a problem in NP-hard, follow these steps:

(i) Show a known NP-hard problem reduces to your problem:- Choose a problem already known to be NP-hard. Demonstrate that solving your problem can also solve the known NP-hard problem.

(ii) Construct a Polynomial-time reduction:- Design a transformation of inputs from the known NP-hard problem to input of problem.

(iii) Argue correctness and efficiency:-

- Prove the reduction is correct.
- Show the reduction process takes polynomial time.

2. what is a NP-Complete Problem?

Ans An NP-complete problem is a problem that satisfies two conditions:

(i) it is in NP - SOP can be verified in polynomial time.

ii) it is NP-hard - Every problem in NP can be reduced to it in polynomial time.

3. Give some examples of NP Problems.

Ans Here are some examples of NP problems:-

(i) Traveling Salesman Problem (TSP): Find the shortest route visiting all cities exactly once.

(ii) Knapsack Problem:

(iii) Boolean Satisfiability problem (SAT)

(iv) Graph Coloring

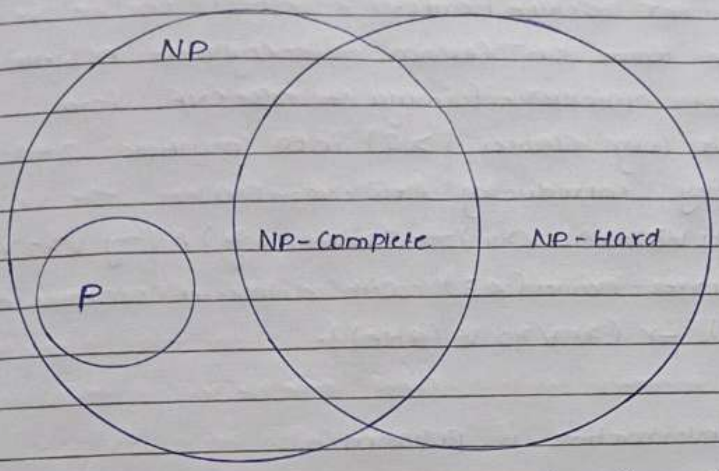
(v) Hamiltonian Cycle

(vi) Subset Sum:- find if there's a subset of numbers that sums to a specific value.

4. Ans

4. Draw the venn diagram of P, NP, NP hard and NP-complete Problems.

Ans



5. What is the time complixity of traveling saleman problem. Is it NP?

Ans The time complexity of Travelling saleman problem depends on the approach used to solve it. like

Brute force :- $O(n!)$, where n = no. of cities visited

dynamic programming :- $O(n^2 \cdot 2^n)$

Yes, TSP is in NP. A Problem is in NP if:

i) A proposed sol. can be verified in polynomial time.

2.) for TSP, given a candidate sol. (a tour), we can compute its total cost and verify whether it is less than given value k in $O(n)$ time.

- Sec-B (3x7 = 21)

6. Prove that 3-CNF SAT Problem is NP-complete.

Ans TO prove that the 3-CNF SAT ~~is in NP~~ problem is NP-complete we need to show two things :-

i) 3-CNF SAT is in NP

ii) 3-CNF SAT is NP-hard

i) 3-CNF SAT is in NP :- A solution (truth assignment)

can be verified in polynomial time by checking if all clauses in the formula are satisfied.

ii) 3-CNF SAT is NP-hard:-

- Reduction from SAT (known NP-complete problem):-
  - Convert any SAT formula to 3-CNF form:
    - Split long clauses (>3) into multiple 3-literal clauses by introducing new variable
      e.g. $(x_1 \vee x_2 \vee x_3 \vee x_4) \rightarrow (x_1 \vee x_2 \vee y) \wedge (\neg y \vee x_3 \vee x_4)$.
    - Pad shorter clauses (<3) with duplicates or constants (e.g. $(x_1 \vee x_2) \rightarrow (x_1 \vee x_2 \vee false)$).

This transformation is polynomial.

so

3-CNF SAT is in NP because SOP can be verified in polynomial time.

3-CNF SAT is NP-hard because SAT reduces to it in polynomial time.

thus, 3-CNF SAT is NP-complete.

7 State and Prove Cook's Theorem.

Ans Cook's Theorem

Statement :-

The Boolean Satisfiability problem (SAT) is NP-complete. In other word, SAT is the first problem that was proven to be NP-complete, and it is both in NP and NP-hard.

formally, Cook's Theorem states:-

- SAT is in NP.
- Every Problem in NP can be reduced to SAT in polynomial time.

Proof outline :-

1. Proof Strategy:- we will show SAT is in NP and is NP-hard
2. Part 1: SAT is in NP:-
   - Given a Boolean formula and an assignment, checking

satisfiability can be done in polynomial time.
- Thus, SAT ∈ NP by definition.

3. PART-2 : SAT is NP-hard
- We will reduce an arbitrary NP problem (eg - circuit satisfiability) to SAT.
- This proves SAT can simulate any problem in NP.

4. Reduction Details:-
→ for any NP problem's computation, construct a Boolean formula representing:
- Initial configuration
- Transition rules
- Final configuration check

→ Formula exists in polynomial time relative to problem's input.

5. Complexity:-
- Reduction is polynomial-time computable
- If SAT formula is satisfiable, original problem has a solution.
- If SAT formula is unsatisfiable, original problem has no solution.

The theorem essentially proves SAT's universality as a computational problem and its central role in complexity theory.

8. What is a Clique? Prove that Clique problem is NP complete.

Ans A clique in a graph is a subset of vertices such that every two vertices in the subset are connected by an edge. A k-clique is a clique with exactly k vertices.

Clique problem:- Given a graph G and an integer k, the clique problem asks whether there is a clique of size k in the graph.

P.T the Clique Problem is NP-Complete:-

i) Clique is in NP:-
A proposed solution (a set of K vertices) can be verified in polynomial time by checking if every pair of vertices in the set is connected by an edge.

ii) Clique is NP-hard:-
We reduce a known NP-complete problem, the vertex cover problem, to the clique problem:-

- Vertex Cover - Given a graph G and an integer k, determine if there is a set of k vertices such that every edge in the graph is incident to at least one vertex in the set.
- A k-vertex cover in G corresponds to a $(|V|-k)$-clique in G'.

Since vertex cover is NP complete, and we can reduce it to the clique problem in polynomial time, this proves that Clique is NP-hard.

Since the Clique Problem is both in NP and NP-hard, so it is NP-complete.

- Sec-C(3x11=33)

9. Prove that subset sum problem is NP-Complete.

Ans Problem definition:-
Given a set of integers $S = \{a_1, a_2, \ldots, a_n\}$ and an integer T, the subset sum problem asks whether there exists a subset of S whose sum is exactly T.

Now, TO prove that Subset sum Problem is NP-Complete, we need to show:-

i) Subset sum is in NP:-
Given a set of integers S and a target sum T, we can

Verify in polynomial time if a given subset of S sums up to T.

ii) Subset sum is NP-hard :-

- We reduce a known NP-Complete problem, like 3-SAT, to subset sum.
- For each variable x_i in 3-SAT, create two numbers: a_i and b_i.
- For each clause, create a number c_j.
- The target sum T is carefully constructed based on the structure of the 3-SAT formula.
- A satisfying assignment for the 3-SAT formula corresponds to a subset of numbers that sums to T.

This reduction shows that if we could solve subset sum efficiently, we could also solve 3-SAT efficiently. Therefore, Subset Sum is NP-hard.

Since, subset sum is both in NP and NP-hard, it is NP-Complete.

10. Graph Coloring Problem is NP-Complete. Prove its NP completeness.

Ans Problem Definition :-

Given a graph $G = (V, E)$ and an integer k, the Graph Coloring problem asks whether the vertices of G can be colored using k colors such that no two adjacent vertices have the same color.

TO P.T the Graph Coloring problem is NP-Complete, we need to show :-

i) Graph Coloring is in NP :-

Given a graph G, to a number of colors k, and an assignment of colors to the vertices of G, we can verify

In polynomial time if the assignment is a valid k-coloring.

ii) Graph coloring is NP-hard:-

- We reduce a known NP-complete problem, like 3-SAT, to Graph Coloring.
- For each variable x_i in 3-SAT, create two vertices: x_i and $\neg x_i$.
- Connect these two vertices with an edge.
- For each clause, create a clique of three vertices, one for each literal in the clause.
- Connect each literal vertex to the corresponding variable vertices.
- The number of colors required for a valid coloring of this graph corresponds to the satisfiability of the 3-SAT formula.

This reduction shows that if we could solve Graph coloring efficiently, we could also solve 3-SAT efficiently. Therefore, Graph Coloring is NP-hard.

Since, Graph Coloring is both in NP and NP-hard, it is NP-Complete.

11. Prove that Hamiltonian cycle is NP-complete problem.

Ans Hamiltonian Cycle problem

Problem Definition - The Hamiltonian cycle problem asks whether a given graph $G = (V, E)$ contains a cycle that visits every vertex exactly once and returns to the starting vertex.

Proof that Hamiltonian Cycle is NP-Complete:-

i) Hamiltonian Cycle is in NP:-
A solution (a cycle visiting all vertices) can be verified in polynomial time by checking that:
- All vertices are visited exactly once.
- The edges in the cycle exist in the graph.
- The cycle starts and ends at the same vertex.

ii) Hamiltonian Cycle is NP-hard:-
- Reduction from 3-SAT or Vertex Cover:-

→ Alternatively, it is common to reduce the hamiltonian path problem (which is NP-complete) to the Hamiltonian cycle problem.
→ Construct a graph where finding a Hamiltonian path corresponds to finding a Hamiltonian cycle by adding an additional vertex connected to all other vertices.

Since Hamiltonian Path is NP-complete and reducible to Hamiltonian cycle in polynomial time, this proves Hamiltonian Cycle is NP-hard.

So, the Hamiltonian cycle problem is both in NP and NP-hard, it is NP-complete.